

1. What is a Data Hazard?

Definition

A **data hazard** occurs when an instruction depends on the result of a previous instruction that has not yet completed its journey through the pipeline. In pipelined processors, multiple instructions are processed simultaneously, each at a different stage. If one instruction needs data that another is still computing, a hazard arises.

Types of Data Hazards

Types

- **RAW (Read After Write):** Instruction reads a register before it's written by an earlier one. (Most common in MIPS)
- **WAR (Write After Read):** Instruction writes to a register after a later instruction has already read it. (Rare in MIPS)
- **WAW (Write After Write):** Two instructions write to the same register; the order matters. (Also rare in MIPS)

2. Step-by-Step Hazard Identification

Instruction Sequence

Given MIPS Instructions

```
lw $2, 0($1)
add $3, $2, $4
sub $5, $3, $2
or  $6, $2, $7
```

How Do Hazards Occur in the Pipeline?

Pipeline Stages

The MIPS pipeline has 5 stages: **IF** (Instruction Fetch), **ID** (Instruction Decode), **EX** (Execute), **MEM** (Memory), **WB** (Write Back). Each instruction moves through these stages, one per clock cycle, with several instructions in the pipeline at once. If an instruction needs a value that a previous instruction is still working on, a hazard occurs.

Let's analyze the hazards for each instruction pair:

Hazard 1: lw \$2, 0(\$1) and add \$3, \$2, \$4

- **lw** writes to 2 in the **WB** stage (cycle 5).
- **add** reads 2 in the **ID** stage (cycle 3).
- **Hazard:** **add** tries to read 2 before **lw** has written the new value. This is a classic load-use RAW hazard.

Cycle	lw 2,0(1)	add 3,2,4
1	IF	
2	ID	IF
3	EX	ID (needs 2)
4	MEM	EX
5	WB (writes 2)	MEM

Hazard 2: add \$3,\$2,\$4 and sub \$5,\$3,\$2

- **add** writes to 3 in **WB** (cycle 6).
- **sub** reads 3 in **ID** (cycle 4).
- **Hazard:** **sub** tries to read 3 before **add** has written the new value. This is a RAW hazard.
- **sub** also needs 2, which is written by **lw** in WB (cycle 5), but **sub** tries to read it in ID (cycle 4). So, another RAW hazard exists on 2.

Cycle	lw 2	add 3	sub 5
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID (reads 3, 2)
5	WB	MEM	EX
6		WB (writes 3)	MEM

Hazard 3: lw \$2,0(\$1) and or \$6,\$2,\$7

- **or** reads 2 in its **ID** stage (cycle 6).
- **lw** writes 2 in **WB** (cycle 5).
- **Hazard:** **or** tries to read 2 just as **lw** is writing it. If the CPU can't read and write the register in the same cycle, this is a RAW hazard.

Cycle	lw 2	or 6
1	IF	
2	ID	
3	EX	
4	MEM	
5	WB (writes 2)	IF
6		ID (reads 2)

Summary Table of Hazards

Summary Table of Hazards					
	From Instr	To Instr	Register	When Written	When Needed
	lw (I1)	add (I2)	2	WB (cycle 5)	ID (cycle 3)
	add (I2)	sub (I3)	3	WB (cycle 6)	ID (cycle 4)
	lw (I1)	sub (I3)	2	WB (cycle 5)	ID (cycle 4)
	lw (I1)	or (I4)	2	WB (cycle 5)	ID (cycle 6)

3. Pipeline Diagram with Forwarding

Instruction Sequence	
1.	lw \$2, 0(\$1)
2.	add \$3, \$2, \$4
3.	sub \$5, \$3, \$2
4.	or \$6, \$2, \$7

What is Forwarding?
Forwarding is a technique used to resolve data hazards by passing results directly from one pipeline stage to another, so the next instruction does not have to wait for the value to be written back to the register file.

Pipeline Diagram (Cycles 1–7)

Instr	1	2	3	4	5	6	7
lw	IF	ID	EX	MEM	WB		
add		IF	ID	EX →	MEM	WB	
sub			IF	ID	EX →	MEM	WB
or				IF	ID	EX →	MEM

Legend
EX → indicates a forwarding path from a previous instruction's pipeline stage to the EX stage of the current instruction.

How Forwarding Resolves Hazards

- When **add** needs 2 (from **lw**), the value is forwarded from the MEM stage of **lw** to the EX stage of **add**, so **add** does not have to wait for **lw** to finish WB.
- When **sub** needs 3 (from **add**), the value is forwarded from the WB or MEM stage of **add** to the EX stage of **sub**.
- When **sub** and **or** need 2 (from **lw**), the value is forwarded from the MEM or WB stage of **lw** to the EX stage of **sub** and ID/EX of **or**.
- This forwarding allows the pipeline to keep moving without stalling, as shown in the diagram above.

4. Pipeline Diagram with Stalls (No Forwarding)

No Forwarding: Why Stalls Occur

If forwarding is not available, the Hazard Detection Unit (HDU) must insert stalls (NOPs) to resolve data hazards. This prevents instructions from using incorrect data, but slows down the pipeline.

Instruction Sequence

Given MIPS Instructions

1. **lw** \$2, 0(\$1)
2. **add** \$3, \$2, \$4
3. **sub** \$5, \$3, \$2
4. **or** \$6, \$2, \$7

Pipeline Diagram with Stalls (Cycles 1–9)

Instr	1	2	3	4	5	6	7	8	9
lw	IF	ID	EX	MEM	WB				
add		IF	NOP	ID	EX	MEM	WB		
sub				IF	NOP	ID	EX	MEM	WB
or						IF	NOP	ID	EX

How the Hazard Detection Unit Resolves Hazards

- When **add** needs 2 (from **lw**), the value is not ready until **lw** completes WB. The HDU inserts a NOP (stall) after IF of **add**, so **add** waits until 2 is available.
- When **sub** needs 3 (from **add**), the value is not ready until **add** completes WB. The HDU inserts a NOP after IF of **sub**, so **sub** waits until 3 is available.
- When **or** needs 2 (from **lw**), the value is not ready until **lw** completes WB. The HDU inserts a NOP after IF of **or**, so **or** waits until 2 is available.
- Each NOP represents a cycle where the pipeline is stalled, and no useful work is done for that instruction.